

C++ Assignment 4 — Move Semantics, Operator Overloading, Inheritance & Polymorphism

Topics: Move Constructor · Move Assignment Operator · Rule of Five · `std::move` · Lvalue vs Rvalue · Operator Overloading (Arithmetic, Relational, Stream `<</>>`, Subscript `[]`, Pre/Post-Increment) · Member vs Non-member Operators · Friend Functions · Friend Classes · Inheritance (Single, Multilevel, Hierarchical) · Constructor/Destructor Execution Order · Function Overriding · `virtual` Keyword · Polymorphism (Compile-Time & Run-Time) · Abstract Class · Pure Virtual Functions · `override` · Virtual Destructor · Upcasting · `dynamic_cast`

Question 1 — Financial Ledger: Move Semantics & Operator Overloading

You are building a financial accounting module for a trading platform. The core class is `LedgerEntry` — it stores a transaction description and a dynamically allocated array of daily amounts (e.g., 30 days of transaction values). The team leader reviews your code and flags two problems:

"First, when we return `LedgerEntry` objects from functions, we're deep-copying megabyte-sized buffers unnecessarily — we need move semantics. Second, our accountants want to write `entry1 + entry2`, `entry1 == entry2`, and `cout << entry1` instead of calling methods manually. Overload those operators."

Design class `LedgerEntry` with:

Private data members:

- `string description`
- `double* amounts` — heap-allocated array of daily amounts
- `int days` — size of the array
- `static int totalEntries` — count of live `LedgerEntry` objects

Required member functions — Rule of Five:

```
// Constructor
LedgerEntry(const string& desc, int days);

// Copy Constructor – deep copy
LedgerEntry(const LedgerEntry& other);

// Move Constructor – transfer ownership, leave source in valid empty state
LedgerEntry(LedgerEntry&& other) noexcept;

// Copy Assignment Operator – deep copy with self-assignment guard
LedgerEntry& operator=(const LedgerEntry& other);

// Move Assignment Operator – transfer ownership with self-assignment guard
LedgerEntry& operator=(LedgerEntry&& other) noexcept;
```

```
// Destructor
~LedgerEntry();
```

Required operator overloads:

```
// Add corresponding daily amounts – return a new LedgerEntry
LedgerEntry operator+(const LedgerEntry& other) const;

// Compare total sum of amounts
bool operator==(const LedgerEntry& other) const;
bool operator>(const LedgerEntry& other) const;

// Subscript operator – access/modify individual day's amount
double& operator[](int index);
const double& operator[](int index) const;

// Stream operators – declared as friend functions
friend ostream& operator<<(ostream& out, const LedgerEntry& entry);
friend istream& operator>>(istream& in, LedgerEntry& entry);
```

main() — Required demonstration:

```
int main() {
    // Objective 1 – Constructor & deep copy
    LedgerEntry jan("January Sales", 5);
    jan[0] = 1200.50; jan[1] = 3400.00; jan[2] = 800.75;
    jan[3] = 2100.00; jan[4] = 650.25;

    LedgerEntry feb("February Sales", 5);
    feb[0] = 900.00; feb[1] = 2200.50; feb[2] = 1750.00;
    feb[3] = 3000.00; feb[4] = 475.50;

    cout << jan << endl;
    cout << feb << endl;

    // Objective 2 – Operator + (sum two ledgers into combined)
    LedgerEntry combined = jan + feb;
    cout << "Combined: " << combined << endl;

    // Objective 3 – Relational operators
    cout << "Jan == Feb : " << (jan == feb ? "Yes" : "No") << endl;
    cout << "Jan > Feb : " << (jan > feb ? "Yes" : "No") << endl;

    // Objective 4 – Move constructor (should NOT deep copy)
    LedgerEntry moved = move(jan);
    cout << "After move, jan.amounts is null: "
         << (/* check jan is empty */ true ? "YES" : "NO") << endl;
    cout << "Moved entry: " << moved << endl;
}
```

```

// Objective 5 – Move assignment
LedgerEntry q1("Q1 Total", 5);
q1 = move(feb); // feb's resources transferred to q1
cout << "Q1 (moved from feb): " << q1 << endl;

// Objective 6 – Static member
cout << "Live LedgerEntry objects: " << LedgerEntry::totalEntries << endl;

return 0;
}

```

Expected Output (sample):

```

[LedgerEntry Created] January Sales (5 days)
[LedgerEntry Created] February Sales (5 days)

January Sales : [1200.50, 3400.00, 800.75, 2100.00, 650.25] Total: 8151.50
February Sales : [900.00, 2200.50, 1750.00, 3000.00, 475.50] Total: 8326.00

Combined      : [2100.50, 5600.50, 2550.75, 5100.00, 1125.75] Total: 16477.50

Jan == Feb : No
Jan > Feb : No

[Move Constructor] Ownership transferred from: January Sales
After move, jan.amounts is null: YES
Moved entry : January Sales : [1200.50, 3400.00, ...] Total: 8151.50

[Move Assignment] Ownership transferred from: February Sales
Q1 (moved from feb) : February Sales : [900.00, ...] Total: 8326.00

Live LedgerEntry objects: 3

[Destructor] Combined destroyed
[Destructor] q1 destroyed
[Destructor] moved destroyed

```

Requirements:

- The Move Constructor must set `other.amounts = nullptr` and `other.days = 0` after transferring ownership
- The Move Assignment must handle self-assignment (`if (this != &other)`) and `delete[]` existing memory before taking ownership
- The subscript `operator[]` must throw `out_of_range` if `index < 0 || index >= days`
- `operator<<` must print all amounts and the running total — it must be a `friend` non-member function
- Print a message in the Move Constructor and Move Assignment to confirm they are called (not the copy versions)

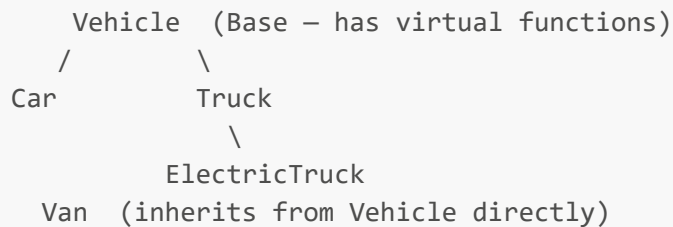
Question 2 — Vehicle Fleet Management: Inheritance & Polymorphism

You are at a logistics company that operates a mixed fleet — cars, trucks, and electric vehicles. The operations team needs a unified reporting system that can process any vehicle type through a common interface, without knowing the exact type at runtime.

The lead architect explains:

"We need a base *Vehicle* class with core attributes. *Car* and *Truck* both inherit from *Vehicle*. *ElectricTruck* inherits from *Truck* — that's multilevel. We also need *Van* inheriting from *Vehicle* separately — hierarchical. The *fuelCost()* and *describe()* functions must be virtual so the dispatch system can call them through a base class pointer without knowing the vehicle type."

Part A — Class Hierarchy Design



Vehicle (Base Class):

```

class Vehicle {
protected:
    string registrationNo;
    string ownerName;
    int    yearOfManufacture;
    double kmDriven;

public:
    Vehicle(const string& reg, const string& owner, int year, double km);

    virtual double fuelCost(double kmToTravel) const = 0; // Pure virtual
    virtual void   describe() const;                    // Virtual – prints
common info
    virtual string vehicleType() const = 0;              // Pure virtual

    virtual ~Vehicle(); // Virtual destructor

    double getKmDriven() const;
    string getRegNo()    const;
};
  
```

Car (Single Inheritance from Vehicle):

- Additional: *string fuelType* ("Petrol" / "Diesel"), *double mileageKmpl*

- `fuelCost(km) = km / mileageKmpl * fuelPricePerLitre` (use `fuelType` to determine price: Petrol = 106.00/L, Diesel = 93.00/L)
- `vehicleType()` returns "Car"
- `describe()` calls `Vehicle::describe()` then adds fuel type and mileage

Truck (Single Inheritance from Vehicle):

- Additional: `double payloadCapacityTons, double fuelEfficiencyKmpl`
- `fuelCost(km)` accounts for payload: efficiency drops 5% per ton carried
- `vehicleType()` returns "Truck"

ElectricTruck (Multilevel — inherits from Truck):

- Additional: `double batteryCapacityKWh, double rangePerChargeKm`
- `fuelCost(km)` uses electricity cost: $(km / rangePerChargeKm) * batteryCapacityKWh * electricityCostPerUnit$ (use 9.50/kWh)
- `vehicleType()` returns "Electric Truck"
- `describe()` must also print battery info

Van (Hierarchical — inherits directly from Vehicle):

- Additional: `int seatingCapacity, double mileageKmpl`
- `fuelCost(km)` = standard petrol calculation
- `vehicleType()` returns "Van"

Part B — Constructor / Destructor Execution Order

In each constructor and destructor, print a message showing which is executing. In `main()`, create a stack object of type `ElectricTruck` and observe the full construction and destruction chain.

```
Expected:
[Vehicle Constructor] V-ET001 : Green Logistics
[Truck Constructor] V-ET001
[ElectricTruck Constructor] V-ET001
...scope ends...
[ElectricTruck Destructor] V-ET001
[Truck Destructor] V-ET001
[Vehicle Destructor] V-ET001
```

Part C — Polymorphic Fleet Report

In `main()`, create a `vector<Vehicle*>` and add at least 5 vehicles of different types (using `new`). Write a function:

```
void printFleetReport(const vector<Vehicle*>& fleet, double tripKm);
```

This function must:

1. Print a formatted table of all vehicles (registration, type, owner, km driven)
2. Call `fuelCost(tripKm)` on each — the correct override must be invoked at runtime
3. Find and print the most fuel-efficient vehicle for the given trip
4. Use `dynamic_cast<ElectricTruck*>` to identify electric trucks and print battery status

Expected Output (sample):

===== FLEET REPORT – Trip Distance: 200 km =====

Reg	Type	Owner	Km Driven
KA01AA001	Car	Ramesh Kumar	45,200
MH04BB002	Truck	Shyam Logistics	1,23,500
GJ07CC003	Electric Truck	Green Fleet Co	89,000

...

===== FUEL / CHARGE COST ESTIMATE =====

KA01AA001 (Car)	: Rs. 1,131.00 (Petrol)
MH04BB002 (Truck)	: Rs. 2,490.00
GJ07CC003 (Electric Truck)	: Rs. 387.50 [Battery: 85% charged]

...

Most Efficient Vehicle: GJ07CC003 (Electric Truck) – Rs. 387.50 for 200 km

Cleanup: Delete all heap-allocated vehicles at the end. Since `Vehicle` has a virtual destructor, the correct derived class destructor will be called through the base pointer — verify this in the output.

Question 3 — Generic Data Processor: Abstract Class & Templates

Your team is building a data processing pipeline for an analytics platform. Different data sources (CSV files, database records, sensor streams) all need to be processed through a common pipeline interface. The system also needs a generic `DataBuffer<T>` class that works for any data type.

"We cannot let anyone instantiate a raw `DataProcessor` — it's a contract, not an implementation. Use a pure virtual interface. And instead of writing separate buffer classes for int, double, and string, use a class template."

Part A — Abstract `DataProcessor` Interface

```
class DataProcessor {
public:
    // Pure virtual functions – all subclasses MUST implement these
    virtual void    loadData(const string& source) = 0;
    virtual void    processData() = 0;
    virtual void    exportResult(const string& destination) = 0;
    virtual string  processorType() const = 0;
    virtual int     recordCount() const = 0;

    // Non-pure virtual – base provides a default report format
    virtual void    printSummary() const;
```

```
virtual ~DataProcessor() = default;
};
```

Implement two concrete processors:

CSVProcessor (inherits from **DataProcessor**):

- **loadData()** — simulates reading a CSV: stores a **vector<string>** of records (hard-code 5 sample rows)
- **processData()** — converts each record to uppercase
- **exportResult()** — prints all processed rows with a **[CSV EXPORT → <destination>]** header
- **processorType()** returns "CSV Processor"
- **recordCount()** returns number of records loaded

SensorStreamProcessor (inherits from **DataProcessor**):

- **loadData()** — simulates reading 8 sensor readings into a **vector<double>** (hard-code values like 23.4, 21.8, 25.1, ...)
- **processData()** — calculates mean, min, max of the readings
- **exportResult()** — prints the statistical summary to **destination**
- **processorType()** returns "Sensor Stream Processor"
- **recordCount()** returns number of readings

In **main()**, use an abstract pointer array:

```
vector<DataProcessor*> pipeline;
pipeline.push_back(new CSVProcessor());
pipeline.push_back(new SensorStreamProcessor());

for (auto* p : pipeline) {
    p->loadData("source_data");
    p->processData();
    p->printSummary();
    p->exportResult("output_dir");
    cout << "---" << endl;
}
```

Verify that attempting **DataProcessor dp;** causes a **compile error** — write it as a commented-out line with an explanation.

Part B — Class Template **DataBuffer<T>**

Write a class template that acts as a fixed-capacity circular buffer (like a ring buffer used in embedded systems and streaming):

```

template <typename T>
class DataBuffer {
private:
    T*   data;           // heap-allocated array
    int  capacity;
    int  head;           // index of oldest element
    int  tail;           // index where next element is written
    int  count;          // current number of elements

public:
    DataBuffer(int capacity);
    ~DataBuffer();

    void push(const T& value); // Add element – overwrites oldest if full
    T    pop();                // Remove and return oldest element
    T    peek() const;         // View oldest without removing
    bool isEmpty() const;
    bool isFull() const;
    int  size() const;

    // Template friend – print all elements in order
    template <typename U>
    friend ostream& operator<<(ostream& out, const DataBuffer<U>& buf);
};

```

In `main()`, demonstrate with three different types:

```

// Integer buffer – simulate sensor tick IDs
DataBuffer<int> tickBuffer(5);
for (int i = 1; i <= 7; i++) tickBuffer.push(i * 10); // Should overwrite oldest
cout << "Tick Buffer: " << tickBuffer << endl;

// Double buffer – simulate temperature readings
DataBuffer<double> tempBuffer(4);
tempBuffer.push(36.6); tempBuffer.push(37.1);
tempBuffer.push(38.2); tempBuffer.push(36.9);
cout << "Before pop: " << tempBuffer << endl;
cout << "Popped: " << tempBuffer.pop() << endl;
cout << "After pop: " << tempBuffer << endl;

// String buffer – simulate log message queue
DataBuffer<string> logBuffer(3);
logBuffer.push("INFO: Server started");
logBuffer.push("WARN: High memory usage");
logBuffer.push("ERROR: DB connection timeout");
logBuffer.push("INFO: Retry successful"); // Overwrites oldest
cout << "Log Buffer: " << logBuffer << endl;

```

Expected Output (sample):

Tick Buffer: [30, 40, 50, 60, 70] (10 and 20 were overwritten)

Before pop: [36.6, 37.1, 38.2, 36.9]

Popped: 36.6

After pop: [37.1, 38.2, 36.9]

Log Buffer: ["WARN: High memory usage", "ERROR: DB connection timeout", "INFO: Retry successful"]

Requirements:

- `DataBuffer` must work for any type `T` — do not write separate classes for `int`, `double`, `string`
- `push()` on a full buffer must overwrite the oldest element (circular behaviour) — not throw an error
- `pop()` on an empty buffer must throw `underflow_error` with message `"Buffer is empty"`
- The template class must be entirely in the header (`.h`) or all in one file — explain in a comment why template definitions cannot be in a separate `.cpp` file

Bonus — `dynamic_cast` Safety Check (Optional)

In `main()` from Part A, after the pipeline loop, add:

```
for (auto* p : pipeline) {
    if (auto* csv = dynamic_cast<CSVProcessor*>(p)) {
        cout << "Found CSV Processor with " << csv->recordCount() << " records."
        << endl;
    } else if (auto* sensor = dynamic_cast<SensorStreamProcessor*>(p)) {
        cout << "Found Sensor Processor — Mean: " << sensor->getMean() << endl;
    }
}
```

Add a `getMean()` getter to `SensorStreamProcessor`. Explain in a comment why `static_cast` would be unsafe here while `dynamic_cast` is safe.